Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

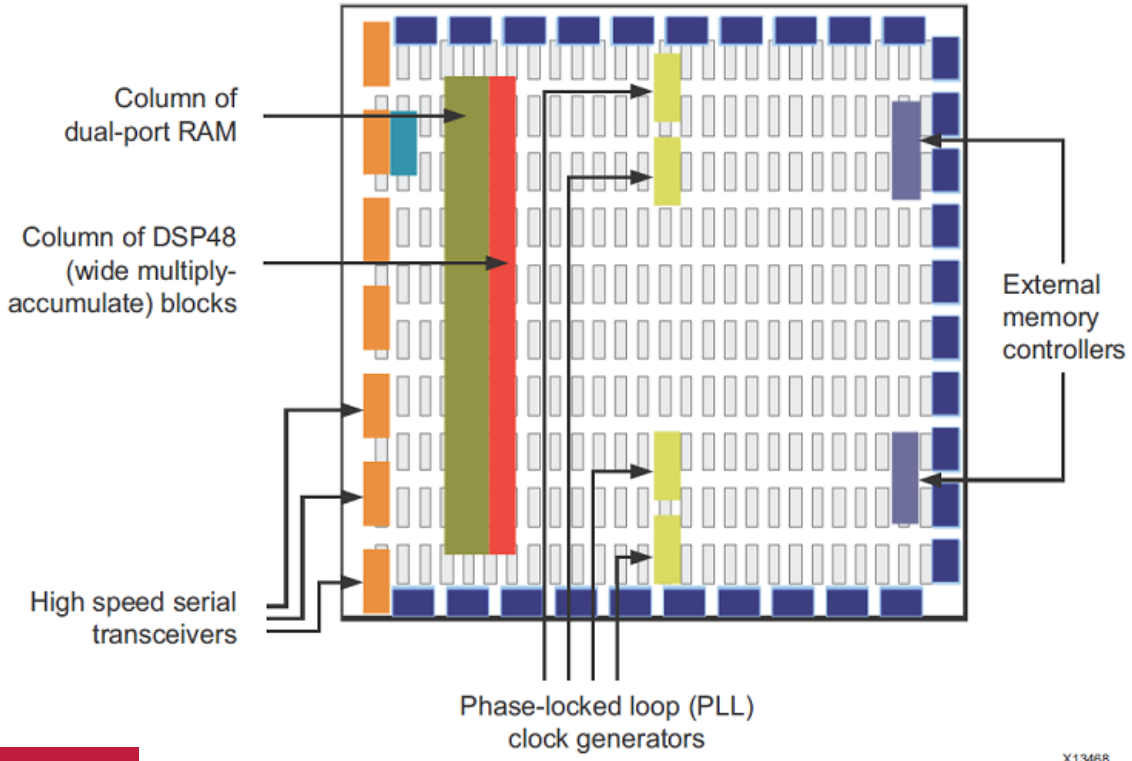
Hardware-Software-Systeme

Kapitel 06 - FPGA-Entwurfstechnik

Prof. Dr.-Ing. Guillermo Payá Vayá, 14. Januar 2025

Weitere Architekturelemente

FPGA Architektur



DSP-Blöcke für typische Signalverarbeitung

- Typische Signalverarbeitungsfunktionen wie Filter, Transformationen, oder Korrelationen, basieren auf den gleichen Grundfunktionen wie

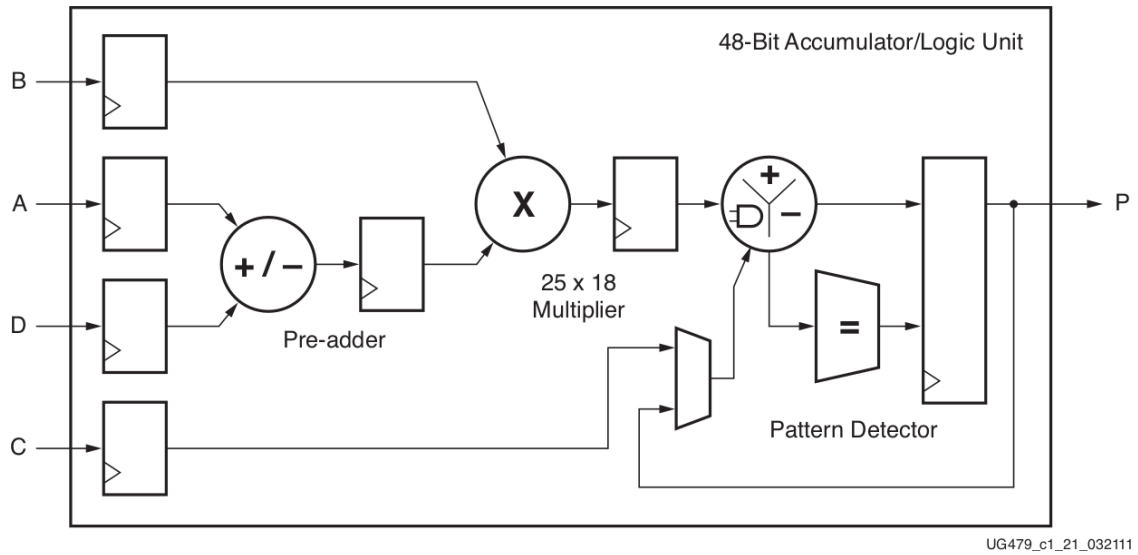
  → ***Multiply-Accumulate-Funktionen***

- sogen. DSP-Blöcke in FPGAs reaslisieren diese häuftig benötigten Grundfunktionen wie
  - Multiplikation
  - Addition/Substraktion
  - Akkumukation

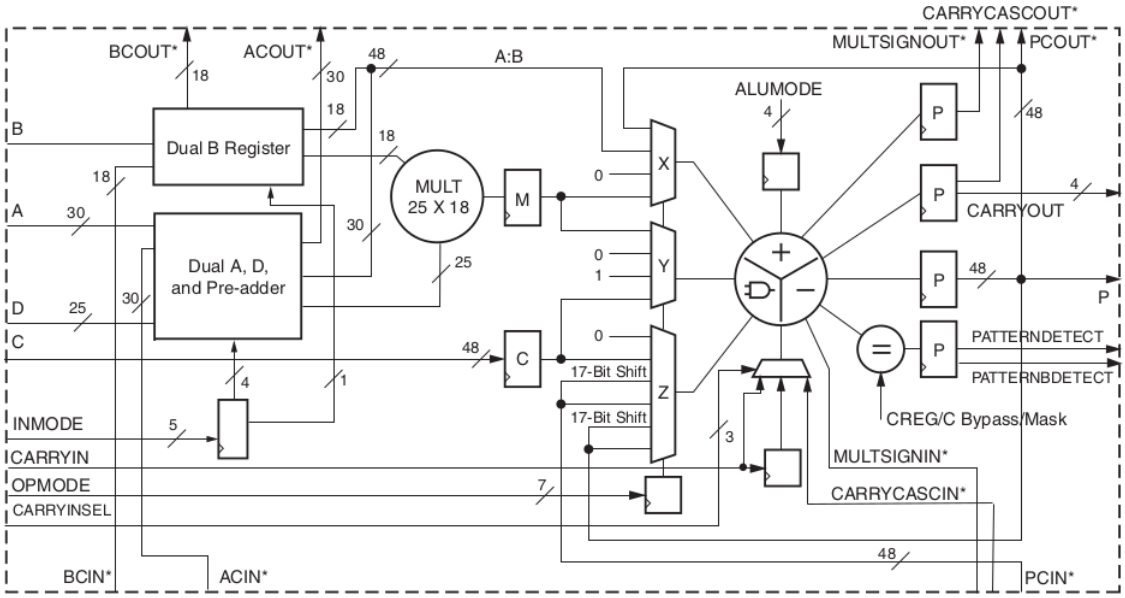
- Diskussion anhand eines Beispiels:

  → ***DSP48E1*** (enhalten in Xilinx Virtex FPGA Bausteinen)
  → ***DSP58*** (enhalten in Xilinx Versal ACAP FPGA Bausteinen)

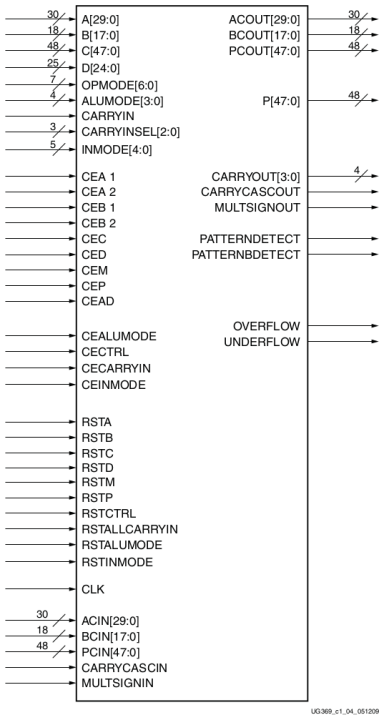
DSP48E1 (Xilinx Virtex FPGA Bausteinen)



Mehr Details ... DSP48E1

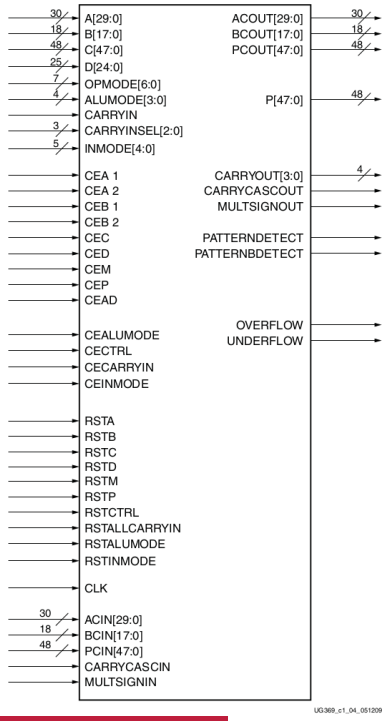


Instanzierung vs. Inferenz (I)




```
architecture rtl of example is
  signal a           : signed(AWIDTH - 1 downto 0);
  signal b           : signed(BWIDTH - 1 downto 0);
  signal c           : signed(CWIDTH - 1 downto 0);
  signal add         : signed(BWIDTH downto 0);
  signal d, mult, p  : signed(BWIDTH + CWIDTH downto 0);
begin
  process(clk)
  begin
    if rising_edge(clk) then
      a <= signed(ain);
      b <= signed(bin);
      c <= signed(cin);
      d <= signed(din);
      if subadd = '1' then
        add <= resize(a, BWIDTH + 1) - resize(b, BWIDTH + 1);
      else
        add <= resize(a, BWIDTH + 1) + resize(b, BWIDTH + 1);
      end if;
      mult <= add * c;
      p <= mult + d;
    end if;
  end process;
  pout <= std_ulogic_vector(p);
end architecture rtl;
```

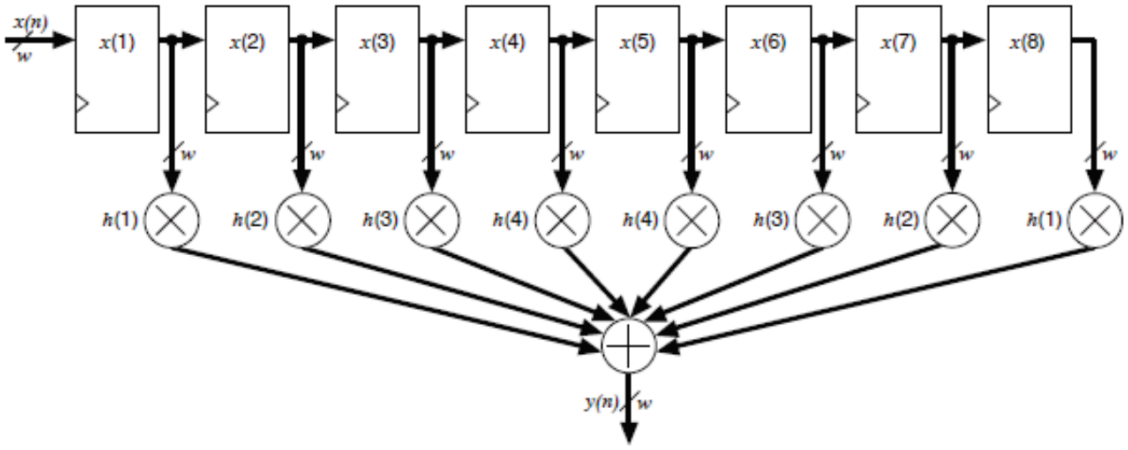

Instanzierung vs. Inferenz (II)




```
alu_dsp_inst : DSP48E1
generic map(
  ACASCREG            => 1,
  ADREG               => 0,
  ALUMODEREG          => 1,
  AREG                => 2,
  AUTORESET_PATDET    => "NO_RESET",
  A_INPUT             => "DIRECT",
  BCASCREG            => 1,
  BREG                => 2,
  B_INPUT             => "DIRECT",
  CARRYINREG          => 1,
  CARRYINSELREG       => 1,
  CREG                => 1,
  DREG                => 0,
  INMODEREG           => 1,
  MASK                => X"000000000000",
  MREG                => 1,
  OPMODEREG           => 1,
  PATTERN             => X"000000000000",
  PREG                => 1,
  SEL_MASK            => "MASK",
  SEL_PATTERN         => "PATTERN",
  USE_DPORT           => FALSE,
  USE_MULT            => "MULTIPLY",
  USE_PATTERN_DETECT  => "PATDET",
  USE_SIMD            => "ONE48")
port map( ... )
```

Effiziente Realisierung von FIR-Filtern (I)

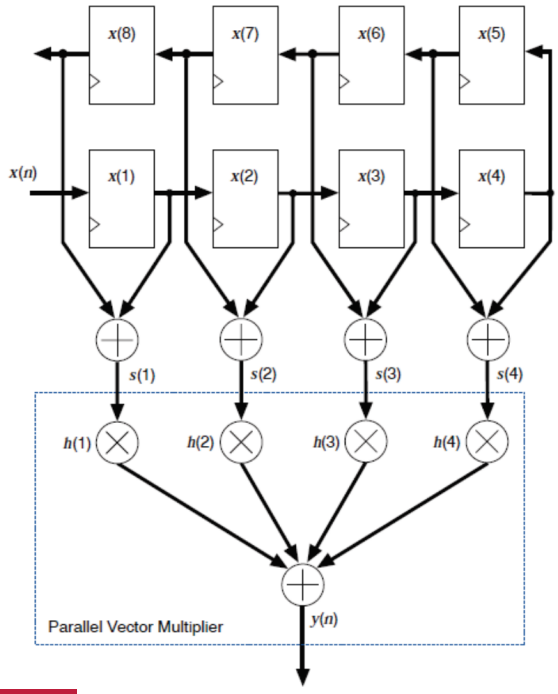
- Transversalfilter in direkter Realisierung



$$y(n) = \sum_{n=1}^{8} x(n)h(n)$$

Effiziente Realisierung von FIR-Filtern (II)

- Addition symmetrischer Taps vor der Multiplikation

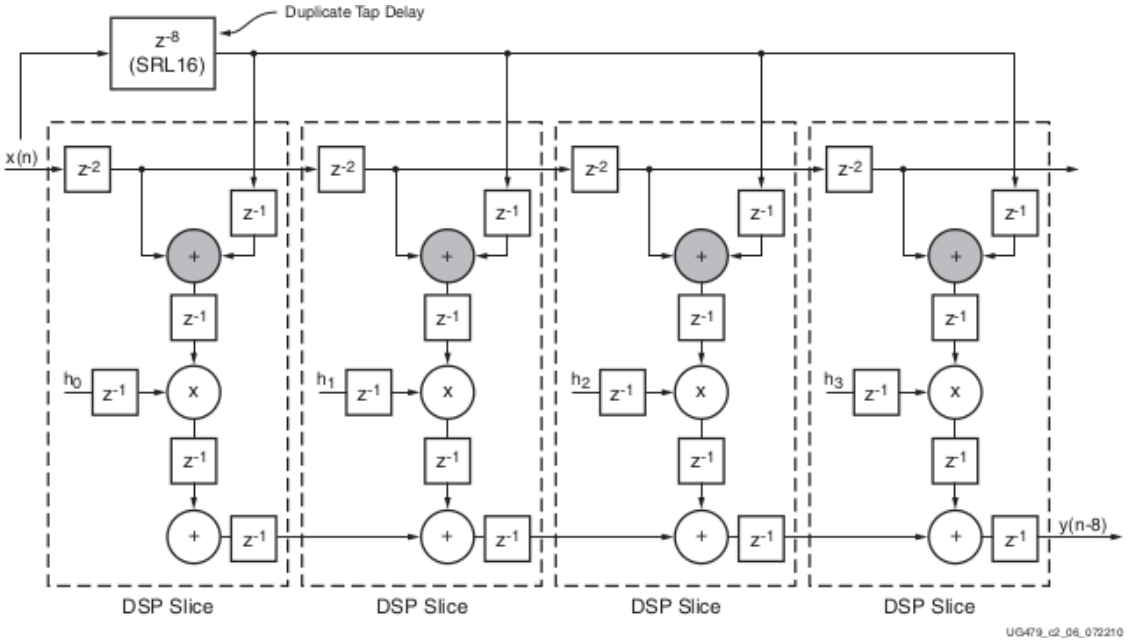


Nutzung der Symmetrie bei linearphasigen FIR-Filtern

⇒ Reduktion der Anzahl der erforderlichen Multiplizierer

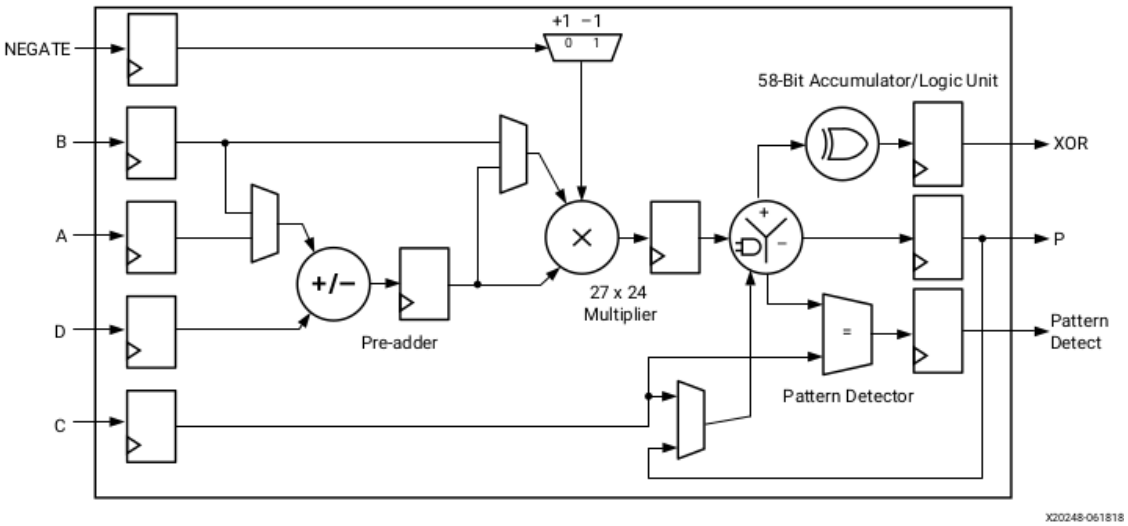
Effiziente Realisierung von FIR-Filtern (III)

- Implementierung mit kaskadiertem DSP48E1

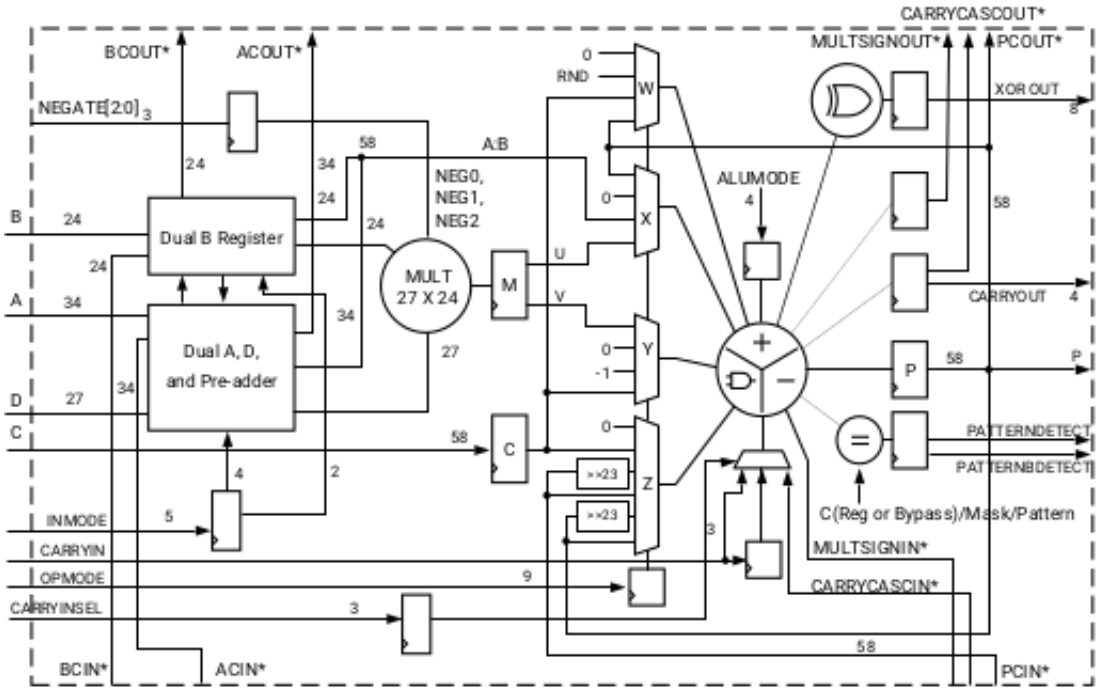


DSP58 (Xilinx Versal ACAP Xilinx)

- ACAP = Adaptive Compute Acceleration Platform
  → Optimiert für künstliche Intelligenz -*KI* (engl. AI inference acceleration)



Mehr Details ... DSP58



*These signals are dedicated routing paths internal to the DSP58 column. They are not accessible through general-purpose routing resources.

X21267-051420

DSP48E1 vs. DSP58

- DSP58 ist die sechste Version des Xilinx DSP.
- Es ist vollständig abwärtskompatibel mit der Architektur DSP48.
- Was ist neu?
  - $27 \times 24 + 58$ Zweierkomplement-Multiplikations-Akkumulator mit 27-Bit-Voraddition und optional Produktverneinung
  - $18 \times 18 + 58$ Zweierkomplement-komplexer Multiplikationsakkumulator mit DSP58
  - Gleitkommaakkumulation mit einfacher Genauigkeit (*binary32*)
  - Gleitkomma-Multiplikationsakkumulator mit gemischter Genauigkeit mit Multiplikanden und Multiplikator statisch und unabhängig wählbar entweder *binary16* oder *binary32*, und *binary32 biasing* und *Akkumulation*

Eingebettete Speicherblöcke (I)

- Eingebettete Speicher werden für eine Vielzahl von Applikationen benötigt:
  - Filter-Delays
  - Schiebregister
  - Datenspeicher
  - Programmspeicher
  - ...
- aktuelle FPGAs unterstützen dies durch unterschiedlich konfigurierbare eingebettete Speicherblöcke (embedded Memories)
- Diskussion anhand eines Beispiels:

  → ***Distribute RAM*** (enhalten in Xilinx Virtex/UltraScale FPGAs)

  → ***Block RAM*** (enhalten in Xilinx Virtex/UltraScale FPGAs)

  → ***Ultra RAM*** (enhalten nur in Xilinx UltraScale FPGAs)

Eingebettete Speicherblöcke (II)

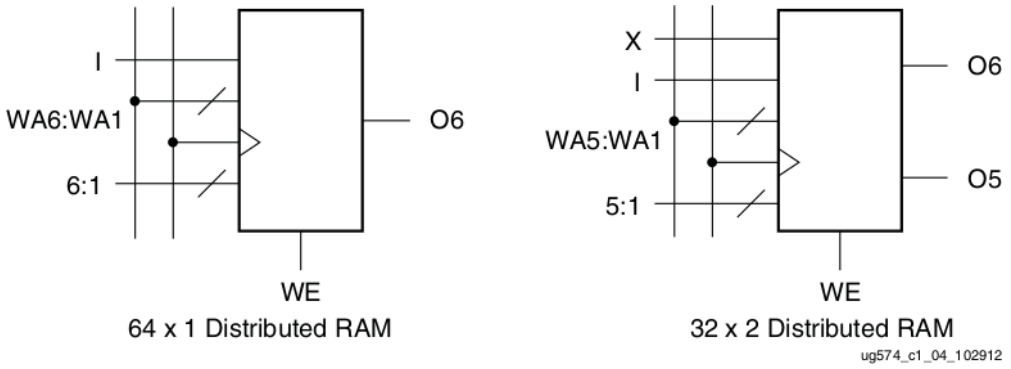
- Die Speicherblöcke sind für unterschiedliche Aufgabenstellungen optimiert:
  - ***Distribute RAM*** (64 bits pro Block)
    - Filter Delay Lines
    - Kleine FIFO Buffer
    - Schieberegister
  - ***Block RAM*** (36Kbits pro Block)
    - General Purpose Speicherblöcke
  - ***Ultra RAM*** (288kbits pro Block)
    - Code-Speicher für Prozessoren
    - Packet Buffering
    - Video Frame Buffer (Bildspeicher)

Eingebettete Speicherblöcke (III)

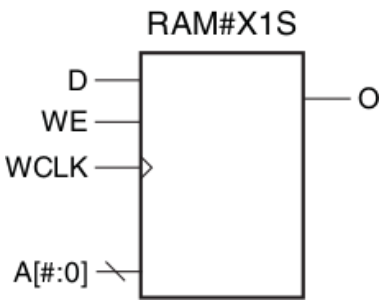
- Jeder Speicherblock kann separat konfiguriert werden.
- Konfigurationsmöglichkeiten:
  - single- oder dual-port RAM
  - FIFO-Buffer
  - ROM
  - Schieberegister
  - mehrere Blöcke eines Typs können zu einem größeren Speicher
  - kombiniert werden
  - Parity Bit Support
  - Byte Enable Support
  - Address Clock Enable Support
  - ...

Distribute RAM (64 bits)

- Die LUT kann entweder als 64 x 1 oder 32 x 2 Speicher konfiguriert werden.
- Nur Single-Port konfiguration ist es moglich
  $\rightarrow$ *Trick: man kann jedoch den Ditributed-RAM-Block duplizieren, um die Anzahl der Leseports praktisch zu erhöhen*



Distributed RAM: Instanzierung vs. Inferenz




```
-- Single-Port RAM with Asynchronous Read (Distributed RAM)
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
entity rams_dist is
  port(
    clk : in  std_ulogic;
    we  : in  std_ulogic;
    a   : in  std_ulogic_vector(5 downto 0);
    di  : in  std_ulogic_vector(15 downto 0);
    do  : out std_ulogic_vector(15 downto 0));
end rams_dist;
architecture syn of rams_dist is
  type ram_type is array (63 downto 0) of
                   std_ulogic_vector(15 downto 0);
  signal RAM : ram_type;
begin
  process(clk)
  begin
    if (clk'event and clk = '1') then
      if (we = '1') then
        RAM(conv_integer(a)) <= di;
      end if;
    end if;
  end process;
  do <= RAM(conv_integer(a));
end architecture syn;
```

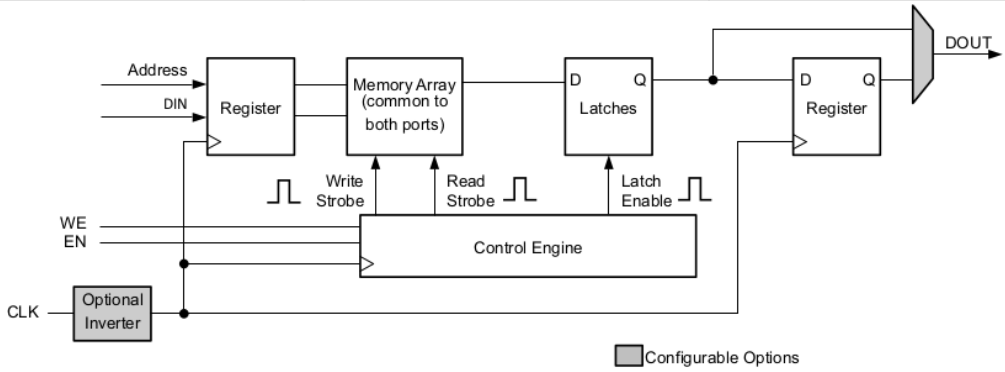

Distributed RAM: Speicherkapazität

RAM	Description	Primitive	Number of LUTs
32 x 1S	Single port	RAM32X1S	1
32 x 1D	Dual port	RAM32X1D	2
32 x 2Q	Quad port	RAM32M	4
32 x 6SDP	Simple dual port	RAM32M	4
64 x 1S	Single port	RAM64X1S	1
64 x 1D	Dual port	RAM64X1D	2
64 x 1Q	Quad port	RAM64M	4
64 x 3SDP	Simple dual port	RAM64M	4
128 x 1S	Single port	RAM128X1S	2
128 x 1D	Dual port	RAM128X1D	4
256 x 1S	Single port	RAM256X1S	4

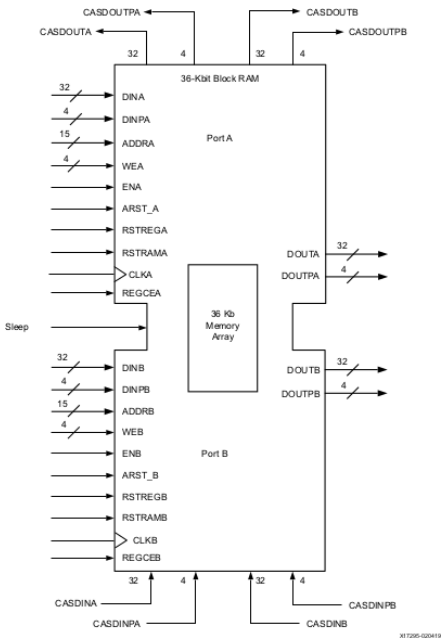
Block RAM (36 Kbits)

- Block-RAM sind eingebettete Komponenten, die als *true* Dual-Port- oder Single-Port-Speicher verwendet werden können.
- Block RAM vs. Distributed RAM:

Action	Distributed RAM	Dedicated Block RAM
Write	Synchronous	Synchronous
Read	Asynchronous	Synchronous



Block RAM: Instanzierung vs. Inferenz




```
-- Dual-Port Block RAM with Two Write Ports
entity rams_tdp_rf_rf is
  port(
    clka, clkb, ena, enb, wea, web : in  std_ulogic;
    addra, addrb        : in  std_ulogic_vector(9 downto 0);
    dia, dib            : in  std_ulogic_vector(15 downto 0);
    doa, dob            : out std_ulogic_vector(15 downto 0));
end rams_tdp_rf_rf;
architecture syn of rams_tdp_rf_rf is
  type ram_type is array (1023 downto 0) of
                    std_ulogic_vector(15 downto 0);
  shared variable RAM : ram_type;
begin
  process(CLKA)
  begin
    if CLKA'event and CLKA = '1' then
      if ENA = '1' then
        DOA <= RAM(conv_integer(ADDRA));
        if WEA = '1' then
          RAM(conv_integer(ADDRA)) := DIA;
        end if;
      end if;
    end if;
  end process;
  process(CLKB)
    ...
  end process;
end architecture syn;
```

Block RAM: Speicherkapazität

Table 1-11: **Port Aspect Ratio for RAMB36E2 (When Used as TDP Memory)**

Port Data Width	Port Address Width	Depth	ADDR Bus	DIN Bus DOUT Bus	DINP Bus DOUTP Bus
1	15	32,768	[14:0]	[0]	NA
2	14	16,384	[14:1]	[1:0]	NA
4	13	8,192	[14:2]	[3:0]	NA
9	12	4,096	[14:3]	[7:0]	[0]
18	11	2,048	[14:4]	[15:0]	[1:0]
36	10	1,024	[14:5]	[31:0]	[3:0]
1 (Cascade)	16	65,536	[15:0]	[0]	NA

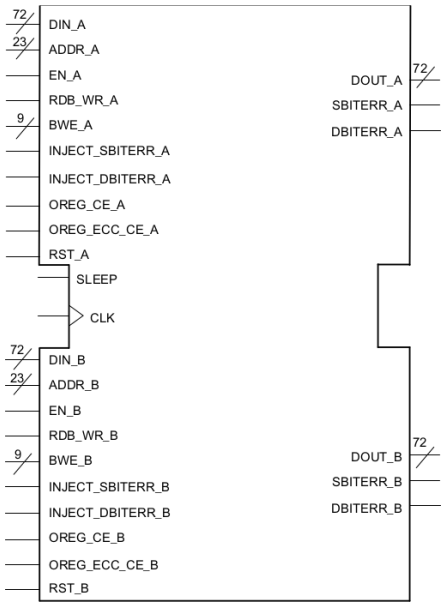
- Mehrere Block-RAMs können kaskadiert werden, um einen größeren zu erstellen.

Ultra RAM (288 Kbits)

- Ultra-RAM sind eingebettete Komponenten, die auch als Dual-Port-oder Single-Port-Speicher verwendet werden können.
- Unterschied zwischen Ultra-RAM und Block-RAM:

Feature	Block RAM	UltraRAM
Clocking	Two clocks	Single clock
Built-in FIFO	Yes	No
Data width	Configurable (1, 2, 4, 9, 18, 36, 72)	Fixed (72-bits)
Modes	SDP and TDP	Two ports, each can independently read or write (a superset of SDP)
ECC	64-bit SECDED Supported in 64-bit SDP only (one ECC decoder for port A and one ECC encoder for port B)	64-bit SECDED One set of complete ECC logic for each port to enable independent ECC operations (ECC encoder and decoder for both ports)
Cascade	• Cascade output only (input cascade implemented via logic resources) • Cascade within a single clock region	• Cascade both input and output (with global address decoding) • Cascade across clock regions in a column • Cascade across several columns with minimal logic resources
Power savings	One mode via manual signal assertion	One mode via manual signal assertion

Ultra RAM: Instanzierung vs. Inferenz



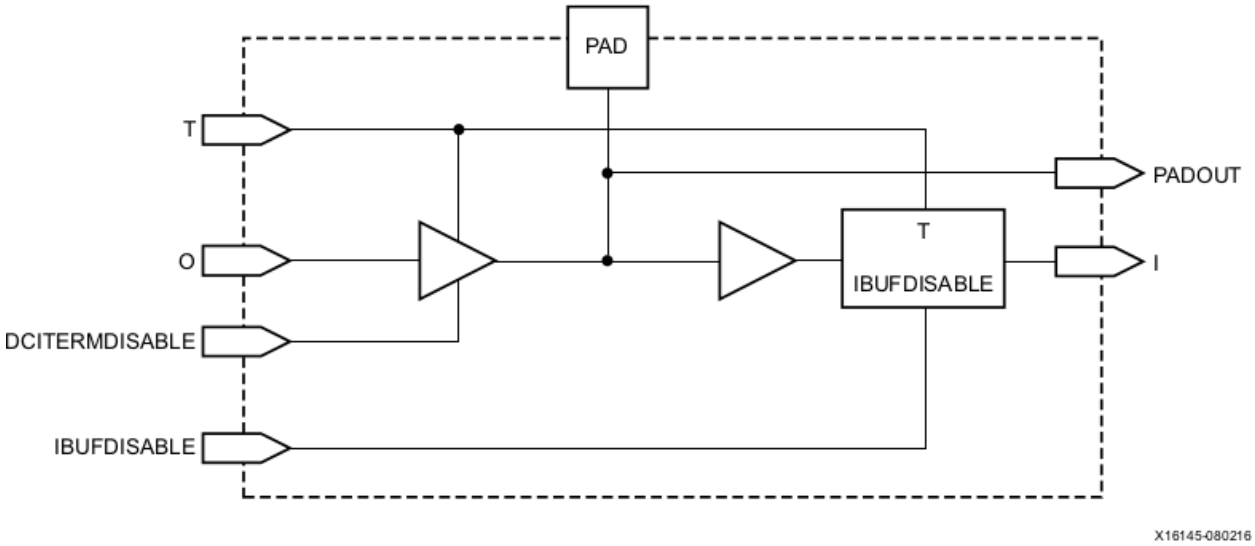

```
-- Dual-Port Ultra-RAM
entity ultra_ram is
  port(
    clk, ena, enb, wea, web : in  std_ulogic;
    addra, addrb            : in  std_ulogic_vector(22 downto 0);
    dia, dib                : in  std_ulogic_vector(71 downto 0);
    doa, dob                : out std_ulogic_vector(71 downto 0));
end ultra_ram;
architecture syn of ultra_ram is
  type ram_type is array (2**23 downto 0) of
                   std_ulogic_vector(71 downto 0);
  shared variable RAM : ram_type;
  attribute ramstyle : string;
  attribute ramstyle of RAM : variable is "ultra";
begin
  process(CLKA)
  begin
    if CLK'event and CLK = '1' then
      if ENA = '1' then
        DOA <= RAM(conv_integer(ADDRA));
        if WEA = '1' then
          RAM(conv_integer(ADDRA)) := DIA;
        end if;
      end if;
      if ENB = '1' then  ... end if;
    end if;
  end process;
end architecture syn;
```


I/O-Architektur - Parameter

- Mit Hilfe der Eingangs-/Ausgangsbaugruppen (I/Os) kommuniziert die Kernlogik mit der Außenwelt
- Die Funktionalität der IOs ist bei FPGAs ebenfalls konfigurierbar.
- Kategorien der Programmierbarkeit von IOs:

Parameter	Beispiele
Richtung	▪ Eingang ▪ Ausgang ▪ bidirektional
Logik	▪ nicht invertierend vs. invertierend ▪ Registerstufe oder Kombinatorik
Signalform	▪ Spannungsebene ▪ Tristate-Fähigkeit ▪ Signalstandard (CMOS, TTL, PCI, GTL, LVDS, etc.)

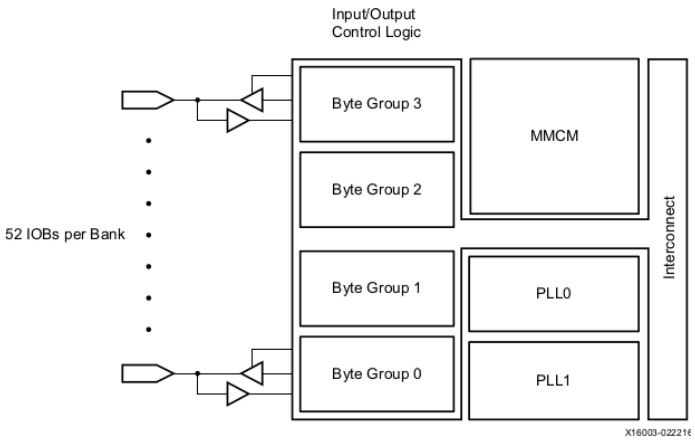
I/O-Architektur - Pin Konfiguration



I/O-Architektur - Blockweise Konfiguration

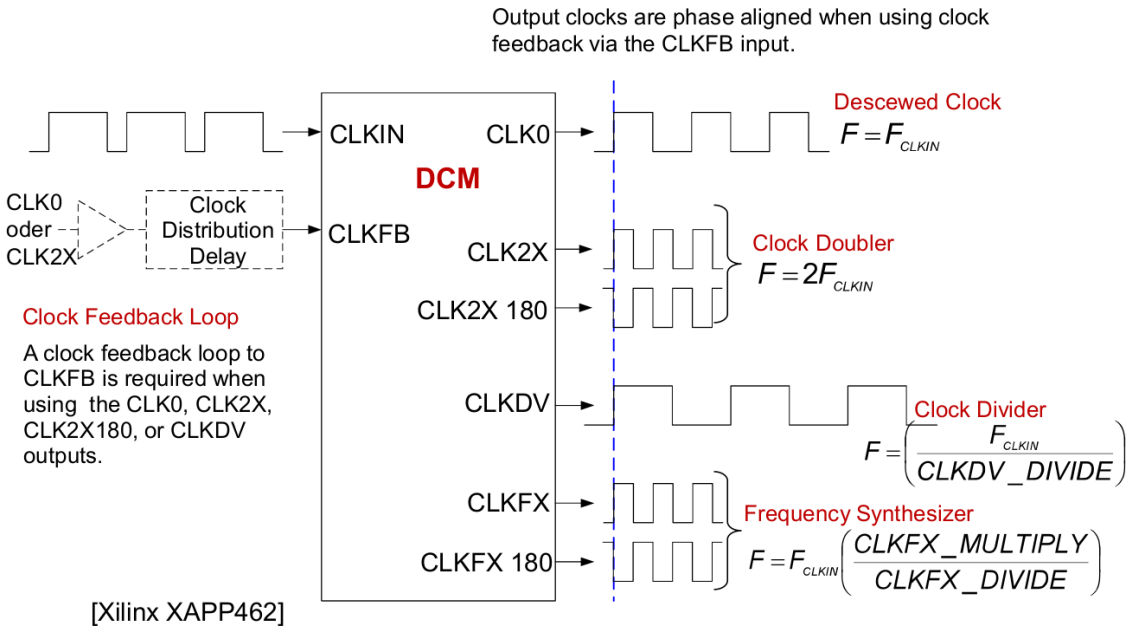
- **Beispiel:** Blockweise Konfigurationsmöglichkeiten der IOs bei Xilinx FPGAs

Feature
3.3V I/O standards(1)
2.5V I/O standards(1)
1.8V I/O standards(1)
1.5V I/O standards(1)
1.35V I/O standards(1)
1.2V I/O standards(1)
1.0V POD I/O standard
LVDS signaling
Digitally-controlled impedance (DCI) and DCI cascading
Internal V_{REF}
Internal differential termination (DIFF_TERM)
IDELAY
ODELAY
IDELAYCTRL
ISERDES
OSERDES
Transmitter pre-emphasis



Clock Synthese - Digital Clock Manager (DCM)

- **Beispiel:** Clock-Synthese-Möglichkeiten beim Xilinx FPGAs



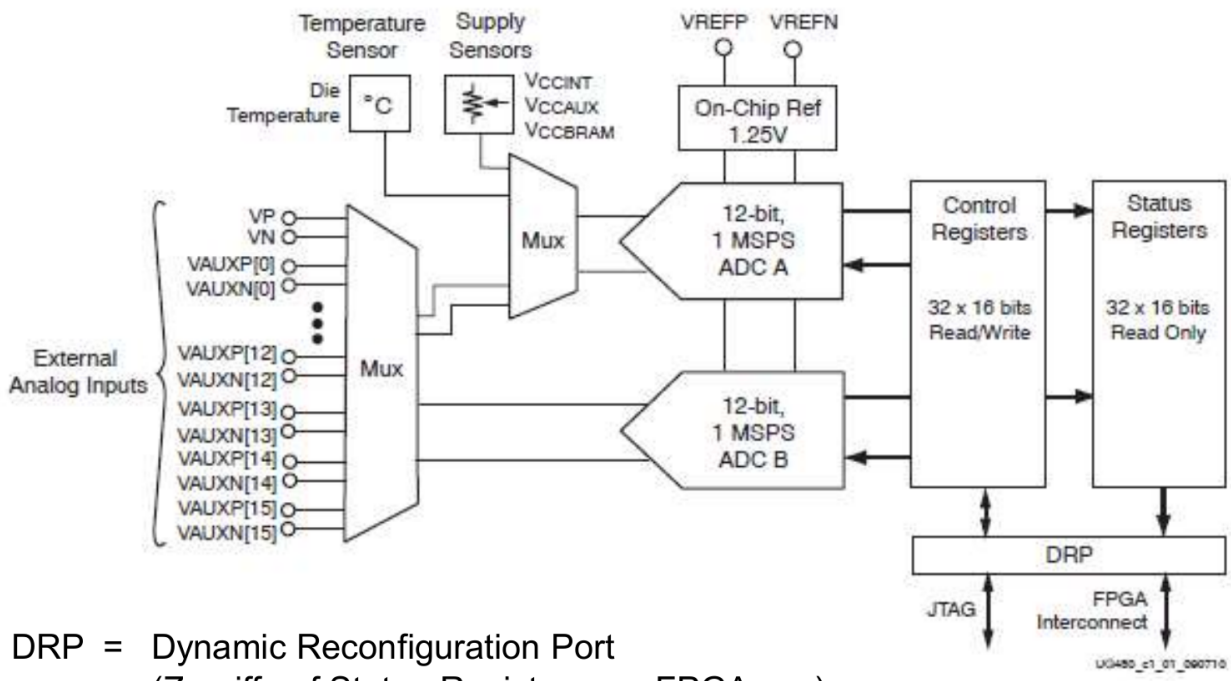
[Xilinx XAPP462]

Analog-Digital-Wandler auf FPGAs

- Viele Anwendungen erfordern den Einsatz von Sensoren für
  - Spannung,
  - Druck,
  - Temperatur, Feuchtigkeit,
  - Gravitationskraft, Beschleunigung, etc.

  $\rightarrow$ Diese Sensoren liefern analoge Messwerte.
- Um diese Signale digital zu verarbeiten werden Analog-Digital- (ADC) und Digital-Analog-Wandler (DAC) benötigt.
- seit 2005 analoge Mixed-Signal-Funktionalitäten auf FPGAs (seit Virtex 5, FPGA System Monitoring)
- seit 2011 Integration von zwei 1 MSPS ADCs mit 12 Bit Auflösung (seit Virtex 7, Artix, Kintex, Zync)
- integriert sind ferner on-Chip-Sensoren für die Die-Temperatur und die on-Chip-Versorgungsspannungen

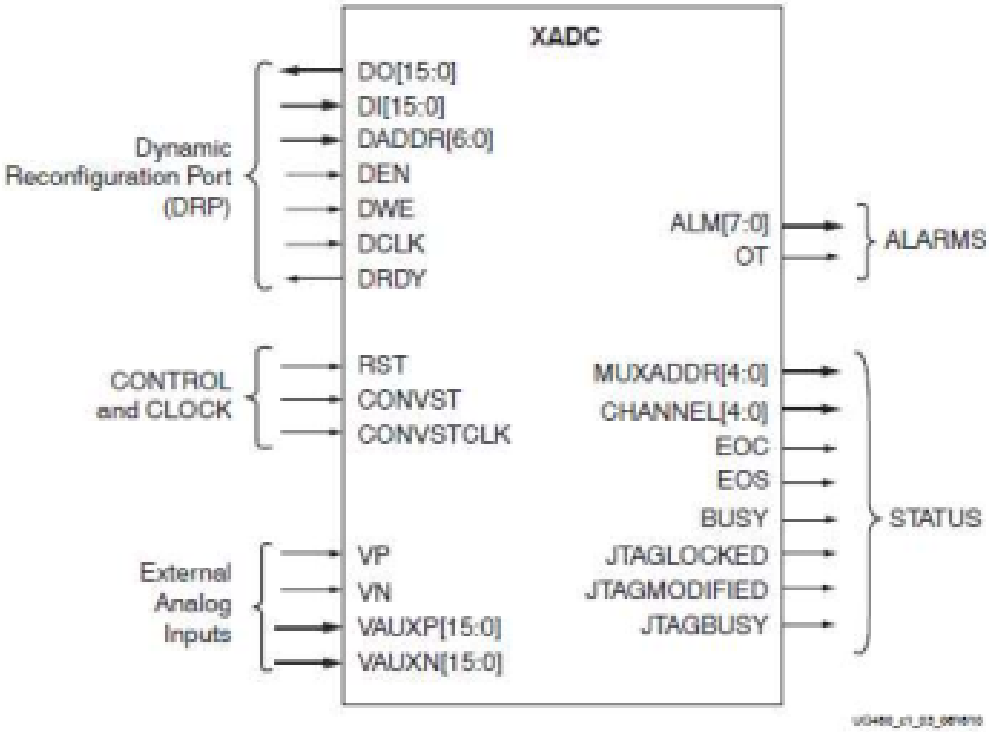
XADC Block-Diagramm



DRP = Dynamic Reconfiguration Port
(Zugriff auf Status-Register vom FPGA aus)

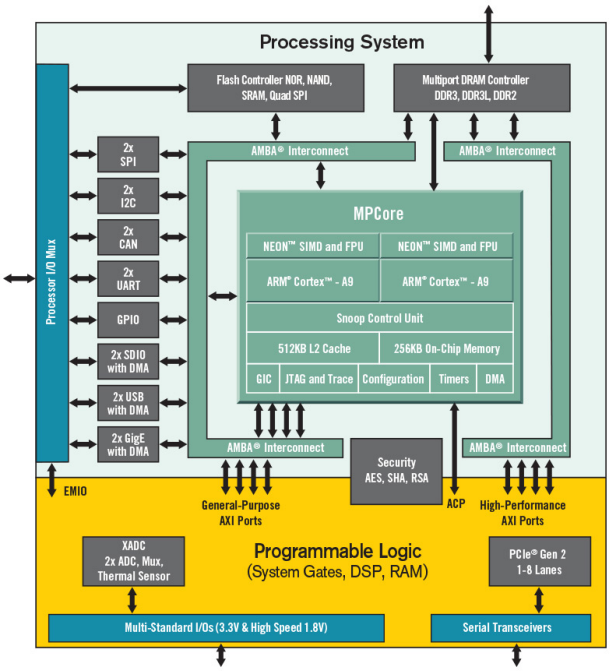
[Xilinx XADC User Guide 2011]

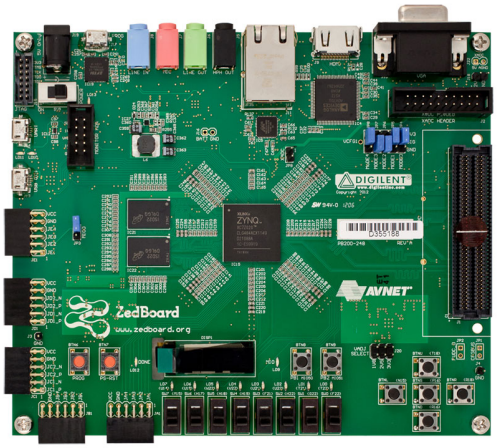
XADC Block-Diagramm



Exemplarische FPGA-SoC

Zynq-7000 All Programmable SoC Overview





FPGA System-on-Chip Familien

Literatur

- AMD/Xilinx Adaptive Computing Documentation Portal, `https://docs.xilinx.com/`
- Frank Kesel, "FPGA Hardware-Entwurf. Schaltungs- und System-Design mit VHDL und C/C++", Walter de Gruyter GmbH, 2018
- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.
- Janick Bergeron, "Writing Testbenches: Functional Verification of HDL Models", Springer Science & Business Media, 2012.